

Job Shop Scheduling: (NP Problem)

Job shop scheduling is an optimization problem that allocates shared machines or resources to a variety of distinct jobs over time. Each job consists of an ordered sequence of specific tasks, and the goal is to sequence these tasks to minimize the total time (makespan) required to complete all jobs.

Solution:

Genetic Algorithm - Permutation of job labels with repetition. (Chromosome Encoding)

Ex: Job B - 6 Operations : Job B appears 6 times in the chromosome

If I see job j, I schedule job j's next operation. Any shuffle of this multiset is automatically a valid schedule. No need of repair step.

//core.py

Reference instance: FT06 (Fisher & Thompson 6x6), fetched from JSPLIB (<https://github.com/tamy0612/JSPLIB/blob/master/instances/ft06>).
Known optimal makespan = 55.

Using Fisher & Thompson 6x6 Instance

FT06_JOBS is a list of 6 jobs, each a list of (machine, duration) tuples

FT06_OPTIMUM is the known best answer from the literature, used later as a reference point to check the GA

Fisher & Thompson 10x10 instance (mt10), also from **JSPLIB**. Known **optimum** = 930.

```
FT06_JOBS = [  
    [(2, 1), (0, 3), (1, 6), (3, 7), (5, 3), (4, 6)],  
    [(1, 8), (2, 5), (4, 10), (5, 10), (0, 10), (3, 4)],  
    [(2, 5), (3, 4), (5, 8), (0, 9), (1, 1), (4, 7)],  
    [(1, 5), (0, 5), (2, 5), (3, 3), (4, 8), (5, 9)],  
    [(2, 9), (1, 3), (4, 5), (5, 4), (0, 3), (3, 1)],  
    [(1, 3), (3, 3), (5, 9), (0, 10), (4, 4), (2, 1)],  
]
```

Chromosome - Solution

chromosome length: 36 | **expected:** 36 . (6 jobs x 6 ops)

chromosome: [0 5 5 0 0 3 4 3 1 0 0 3 4 1 1 1 5 3 1 2 3 2 5 2 1 0 2 4 2 4 4 3 4 5 2 5]

Read: Do job 0 first, do job 5 first, do job 5 second, do job 0 second

n_jobs: 6

n_ops_per_job: [6, 6, 6, 6, 6, 6]

chrom_len: 36

n_machines: 6

before shuffle: [0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5,

5, 5]

after shuffle: [0, 5, 5, 0, 0, 3, 4, 3, 1, 0, 0, 3, 4, 1, 1, 1, 5, 3, 1, 2, 3, 2, 5, 2, 1, 0, 2, 4, 2, 4, 4, 3, 4, 5, 2, 5]

def decode():

next_op[j]: which operation number is job j up to next

machine_free[m]: what time does machine m become available (initial it is time 0)

job_free[j]: what time did job j last's operation complete

return max(job_free): tracks each job's latest completion time, updated every time its operation finishes

step 0: job 0 op 0 -> machine 2 (dur 1) | starts 0, finishes 1
step 1: job 5 op 0 -> machine 1 (dur 3) | starts 0, finishes 3
step 2: job 5 op 1 -> machine 3 (dur 3) | starts 3, finishes 6
step 3: job 0 op 1 -> machine 0 (dur 3) | starts 1, finishes 4
step 4: job 0 op 2 -> machine 1 (dur 6) | starts 4, finishes 10
step 5: job 3 op 0 -> machine 1 (dur 5) | starts 10, finishes 15
step 6: job 4 op 0 -> machine 2 (dur 9) | starts 1, finishes 10
step 7: job 3 op 1 -> machine 0 (dur 5) | starts 15, finishes 20
step 8: job 1 op 0 -> machine 1 (dur 8) | starts 15, finishes 23
step 9: job 0 op 3 -> machine 3 (dur 7) | starts 10, finishes 17
step 10: job 0 op 4 -> machine 5 (dur 3) | starts 17, finishes 20
step 11: job 3 op 2 -> machine 2 (dur 5) | starts 20, finishes 25

final makespan: 87 (fitness function)

def tournament_select():

 picks one parent chromosome for the next generation. It grabs k=3 random chromosomes from the population and returns whichever of those 3 has the lowest makespan (best fitness).

Example:

fitnesses: [87, 98, 85, 82, 89]

idx = rng.integers(0, len(pop), size=k) # k=3 Picks 3 random candidates

winner makespan: 82

indices drawn into this tournament: [4 3 2]

their fitnesses: [89, 82, 85]

Index 3 (fitness 82, the best of all 5) was one of the 3 randomly drawn this time, and correctly won its tournament.

Indices 1 (fitness 98, the worst of all 5) and 4 (fitness 89) — even if either was drawn — could never have won against 82.

Index 0 (87) and index 2 (85) — if either was drawn alongside index 3, they'd have lost too, since both are higher than 82.

def pox_crossover():

 combines two parent chromosomes into one child, splits two jobs into two random groups - one group's operations come from parent 1, the other groups come entirely from parent 2

```
parent1: [1 0 1 1 5 0 4 2 2 3 2 0 1 4 3 3 5 4 4 5 5 2 3 5 4 0 4 2 1 3 0 0 1 5 3 2]
parent2: [4 5 2 1 0 3 1 4 4 2 4 1 2 3 2 4 5 3 5 0 3 1 2 0 0 2 3 1 0 3 5 4 5 5 1 0]
child:   [4 5 2 1 0 1 4 4 2 3 4 1 2 2 3 3 4 5 5 0 1 2 3 0 0 2 1 0 5 3 4 5 5 1 3 0]
child gene counts: Counter({4: 6, 5: 6, 2: 6, 1: 6, 0: 6, 3: 6})
```

```
j1 (from parent1) = {3}
everyone else (from parent2) = {0, 1, 2, 4, 5}
```

```
positions of job 3 in child: [9, 14, 15, 22, 29, 34]
positions of job 3 in parent1: [9, 14, 15, 22, 29, 34]
identical positions? True
```

```
child's job-0 genes, in order: [0, 0, 0, 0, 0, 0]
parent2's job-0 genes, in order: [0, 0, 0, 0, 0, 0]
identical positions? True
```

```
first 15 non-job-3 genes in child: [4, 5, 2, 1, 0, 1, 4, 4, 2, 4, 1, 2, 2, 4, 5]
first 15 non-job-3 genes in parent2: [4, 5, 2, 1, 0, 1, 4, 4, 2, 4, 1, 2, 2, 4, 5]
match? True
```

def swap_mutation():

with some probability (rate) , it picks two random positions in the chromosome and swaps their genes.

Example:

```
rng_mut = np.random.default_rng(0)
original = inst.random_chromosome(np.random.default_rng(5))
mutated = swap_mutation(original, rate=1.0, rng=rng_mut)
```

```
original: [2 1 0 1 5 4 2 4 1 3 5 5 1 3 5 5 3 4 0 4 1 4 2 4 0 2 5 3 3 2 3 0 0 2 1 0]
mutated:  [2 1 0 1 5 4 2 4 1 0 5 5 1 3 5 5 3 4 3 4 1 4 2 4 0 2 5 3 3 2 3 0 0 2 1 0]
same gene counts: True
positions that changed: [9, 18]
```

def population_diversity():

we compare similarity between different chromosomes, we use difference as the distance - every unmatched gene counts as one distance. We then find the average of the unique pairs and normalize the result.

APPLYING FUZZY:

```
// fuzzy.py
```

def memberships():

takes one number (x) and returns three numbers to see how much it counts as Low, Medium and High (**0-1**). These triangles peak at different points

shape:

$x = 0.0$ (minimum possible):

$\text{low} = (0.5 - 0)/0.5 = 1.0 \rightarrow \text{fully Low}$

$\text{med} = 1 - |0 - 0.5|/0.5 = 1 - 1 = 0.0 \rightarrow \text{not Medium at all}$

$\text{high} = (0 - 0.5)/0.5 = -1.0 \rightarrow \text{clamped to } 0.0 \rightarrow \text{not High at all}$

$x = 0.5$ (exact middle):

$\text{low} = (0.5 - 0.5)/0.5 = 0.0$

$\text{med} = 1 - |0.5 - 0.5|/0.5 = 1 - 0 = 1.0 \rightarrow \text{fully Medium}$

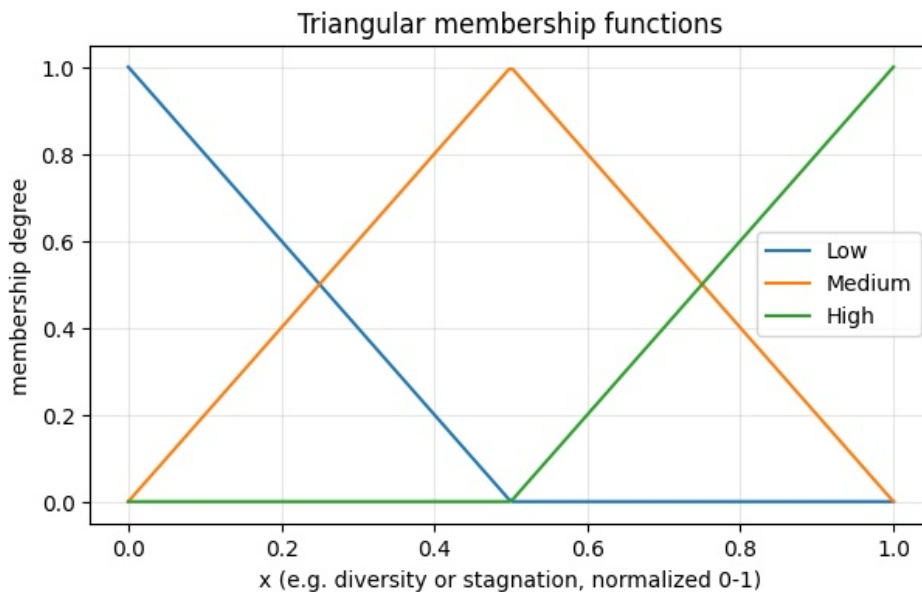
$\text{high} = (0.5 - 0.5)/0.5 = 0.0$

$x = 0.75$ (between Medium and High):

$\text{low} = (0.5 - 0.75)/0.5 = -0.5 \rightarrow \text{clamped to } 0.0$

$\text{med} = 1 - |0.75 - 0.5|/0.5 = 1 - 0.5 = 0.5 \rightarrow \text{half Medium}$

$\text{high} = (0.75 - 0.5)/0.5 = 0.5 \rightarrow \text{half High}$



RULES = {

 ("Low", "Low"): "Medium",

 ("Low", "Medium"): "High",

 ("Low", "High"): "High",

 ("Medium", "Low"): "Low",

 ("Medium", "Medium"): "Medium",

 ("Medium", "High"): "High",

 ("High", "Low"): "Low",

 ("High", "Medium"): "Low",

 ("High", "High"): "Medium",

}

Low diversity + High stagnation → push mutation High

High diversity + Low stagnation → keep mutation Low

takes how diverse the population is, and how stagnant the search is and returns a final mutation rate, with a record of why

fired, num, den: fired will collect details on every rule that mattered. num and den are numerator and denominator of a weighted average

the for loops over all 9 entries in RULES and gives a recommendation

strength: a measure of how true the rules is

it looks up how much current situation matches with the 9 rules, then weigh each rule's suggested mutation rate by how strongly it applies, then average them all together, return the final number with a trail of what influenced it.

Example:

rate, fired = infer(diversity=0.2, stagnation_norm=0.8) [population fairly converged, search closely stuck]

mutation rate: 0.10444444444444445

rules that fired:

```
{'diversity_term': 'Low', 'diversity_deg': 0.6, 'stagnation_term': 'High', 'stagnation_deg': 0.6000000000000001, 'consequent': 'High', 'strength': 0.6}
```

```
{'diversity_term': 'Medium', 'diversity_deg': 0.4, 'stagnation_term': 'High', 'stagnation_deg': 0.6000000000000001, 'consequent': 'High', 'strength': 0.4}
```

```
{'diversity_term': 'Low', 'diversity_deg': 0.6, 'stagnation_term': 'Medium', 'stagnation_deg': 0.3999999999999999, 'consequent': 'High', 'strength': 0.3999999999999999}
```

```
{'diversity_term': 'Medium', 'diversity_deg': 0.4, 'stagnation_term': 'Medium', 'stagnation_deg': 0.3999999999999999, 'consequent': 'Medium', 'strength': 0.3999999999999999}
```

Gen 5: mutation set to 0.104 [raw diversity=0.20, stagnation=0.80] because diversity is Low (0.60) AND stagnation is High (0.60) → mutation should be High (rule strength 0.60); also, diversity is Medium (0.40) AND stagnation is High (0.60) → mutation should be High (rule strength 0.40).

```
//experiment.py
```

```
def crisp_rate():
```

To answer if fuzziness itself actually help or would same 9 rule table work just as same with hard cutoffs

it replaces memberships with hard thresholds

Example:

```
print("crisp_rate(0.2, 0.8):", crisp_rate(0.2, 0.8))
```

```
print("fuzzy infer same inputs:", rate) # reusing `rate` from your earlier infer() call
```

crisp rate(0.2, 0.8): 0.12

fuzzy infer same inputs: 0.10444444444444445

```
result = run_ga(method="fuzzy", seed=42, instance="ft06", n_generations=30, record_trace=True)
```

final makespan: 55 [best]

Known optimal makespan = 55.

best curve: [63 63 61 61 58 58 57 57 55 55 55 55 55 55 55 55 55 55 55 55 55 55 55]

55 55 55 55 55 55 55]

First 3 generations:

Gen 1: mutation set to 0.020 [raw diversity=0.83, stagnation=0.00] because diversity is High (0.67) AND stagnation is Low (1.00) → mutation should be Low (rule strength 0.67); also, diversity is Medium (0.33) AND stagnation is Low (1.00) → mutation should be Low (rule strength 0.33).

Gen 2: mutation set to 0.022 [raw diversity=0.83, stagnation=0.05] because diversity is High (0.65) AND stagnation is Low (0.90) → mutation should be Low (rule strength 0.65); also, diversity is Medium (0.35) AND stagnation is Low (0.90) → mutation should be Low (rule strength 0.35).

Gen 3: mutation set to 0.020 [raw diversity=0.82, stagnation=0.00] because diversity is High (0.63) AND stagnation is Low (1.00) → mutation should be Low (rule strength 0.63); also, diversity is Medium (0.37) AND stagnation is Low (1.00) → mutation should be Low (rule strength 0.37).

Last 3 generations (should show high stagnation, since stuck at optimum):

Gen 28: mutation set to 0.114 [raw diversity=0.36, stagnation=0.95] because diversity is Medium (0.71) AND stagnation is High (0.90) → mutation should be High (rule strength 0.71); also, diversity is Low (0.29) AND stagnation is High (0.90) → mutation should be High (rule strength 0.29).

Gen 29: mutation set to 0.120 [raw diversity=0.34, stagnation=1.00] because diversity is Medium (0.68) AND stagnation is High (1.00) → mutation should be High (rule strength 0.68); also, diversity is Low (0.32) AND stagnation is High (1.00) → mutation should be High (rule strength 0.32).

Gen 30: mutation set to 0.120 [raw diversity=0.32, stagnation=1.00] because diversity is Medium (0.63) AND stagnation is High (1.00) → mutation should be High (rule strength 0.63); also, diversity is Low (0.37) AND stagnation is High (1.00) → mutation should be High (rule strength 0.37).

```
curves_static, finals_static = run_many("static", n_runs=15, seed0=1000, instance="ft06",
n_generations=150)
curves_crisp, finals_crisp = run_many("crisp", n_runs=15, seed0=1000, instance="ft06",
n_generations=150)
curves_fuzzy, finals_fuzzy = run_many("fuzzy", n_runs=15, seed0=1000, instance="ft06",
n_generations=150)
```

static final makespans: [57 59 59 59 59 55 59 57 57 58 59 55 59 58 58]

crisp final makespans: [55 59 55 55 59 55 59 55 55 58 59 57 55 58 59]

fuzzy final makespans: [55 59 57 55 60 55 59 55 55 59 59 55 59 58 59]

static mean: 57.86666666666667 | **crisp mean:** 56.86666666666667 | **fuzzy mean:** 57.26666666666667

```
t_stat, p_val = stats.ttest_ind(finals_fuzzy, finals_crisp)
print(f"\nfuzzy vs crisp: t={t_stat:.3f}, p={p_val:.3f}")
t_stat2, p_val2 = stats.ttest_ind(finals_fuzzy, finals_static)
print(f"fuzzy vs static: t={t_stat2:.3f}, p={p_val2:.3f}")
```

static std: 1.359738536958076 | **crisp std:** 1.8208667044996887 | **fuzzy std:** 1.9482185594936607

fuzzy vs crisp: $t=0.561$, $p=0.579$
fuzzy vs static: $t=-0.945$, $p=0.353$

NEED TO TRY WITH LARGER PROBLEM TO SEE ACCURACY